

Chapter 1: Image Classification with Linear Classifiers

Basics and Applications of Deep Learning

Fall 2026

Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



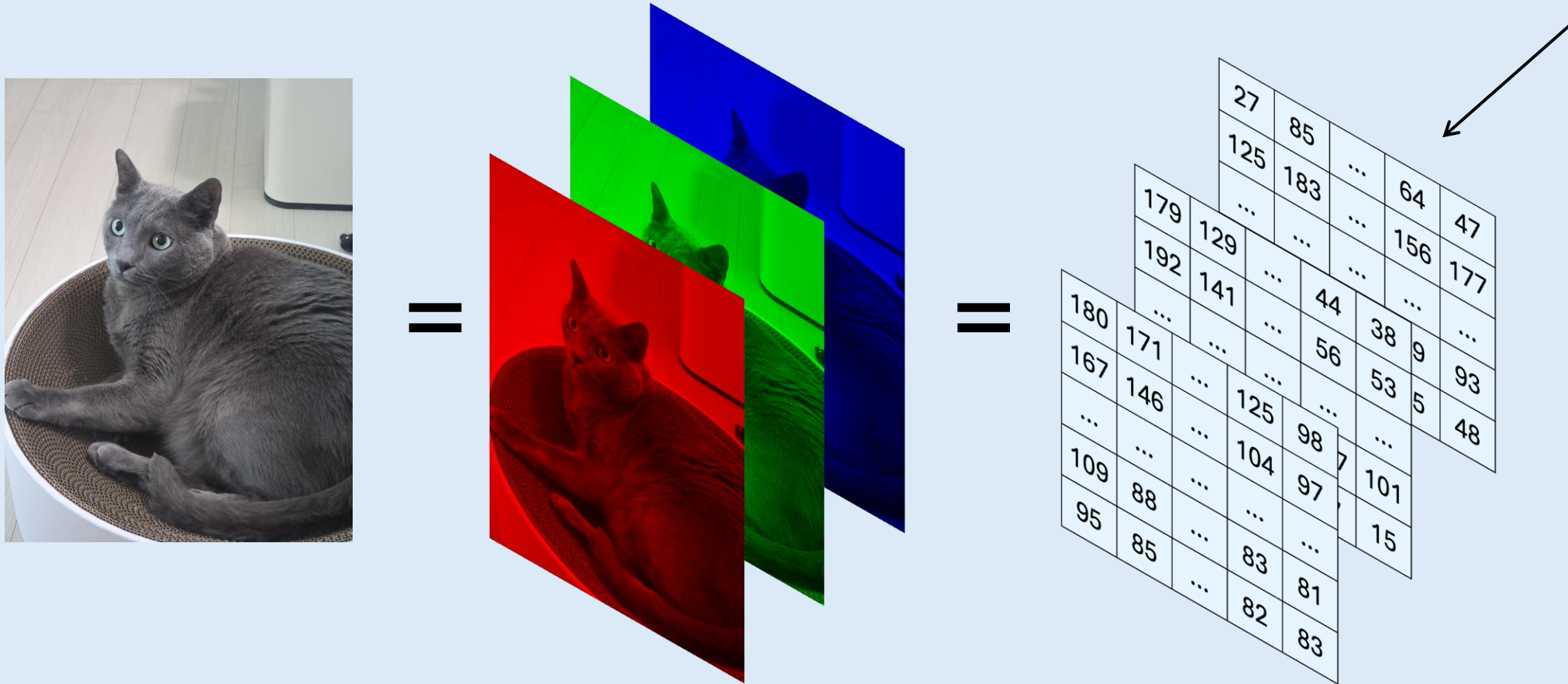
Image classification: a core task in computer vision



(assume given a set of possible labels)
{dog, cat, car, ship, ...}

—————→ **cat**

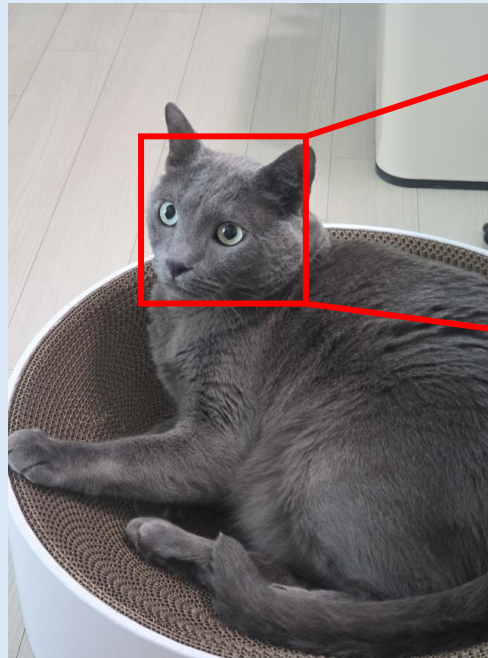
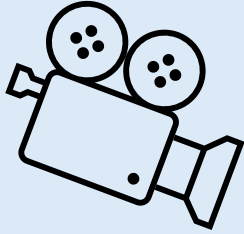
The problem: semantic gap



An image is a tensor of integers
between [0, 255]:

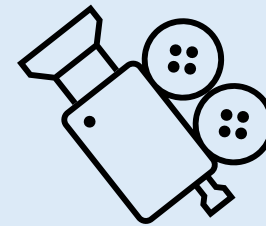
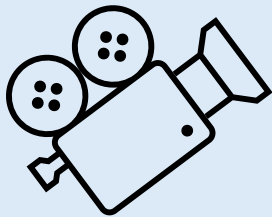
e.g. 800 x 600 x 3 (3 channels RGB)

Challenges: viewpoint variation

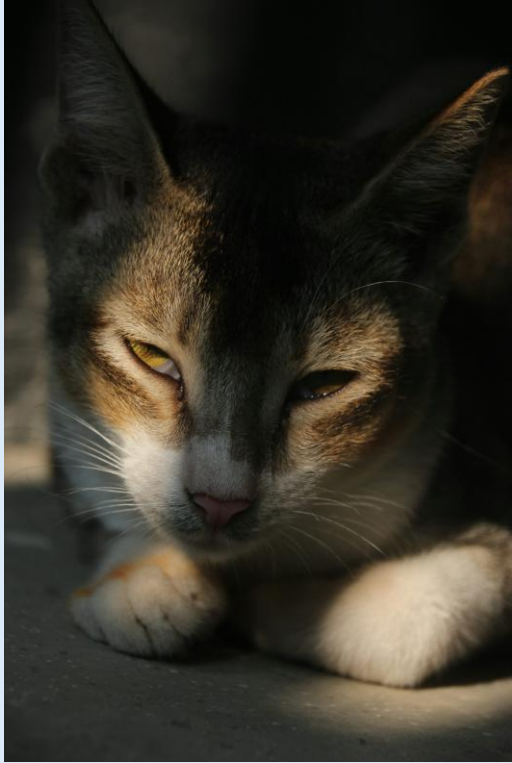


179	129	...	44	38
192	141	...	56	53
...	...	...	...	...
49	180	...	127	101
62	51	...	37	15

All pixels change when
the camera moves!



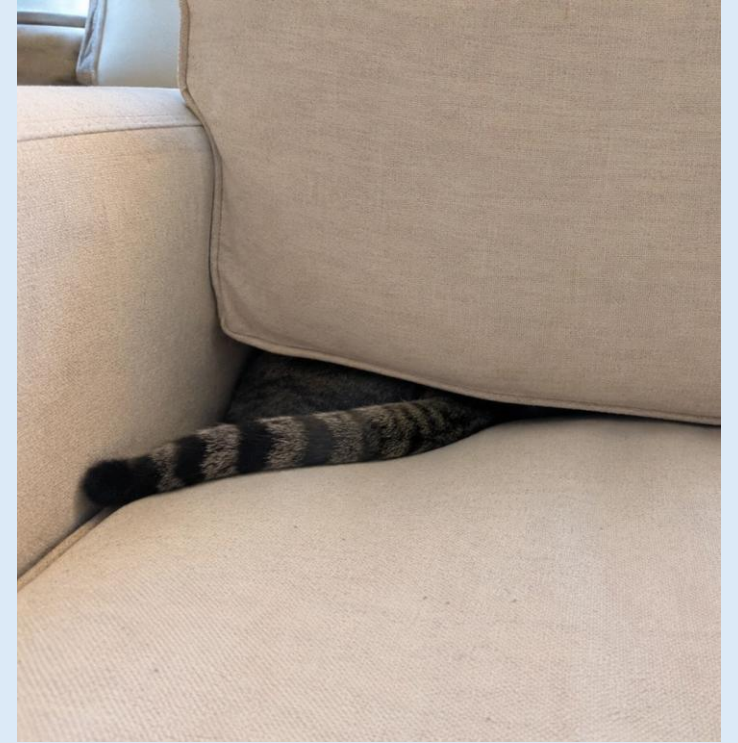
Challenges: illumination



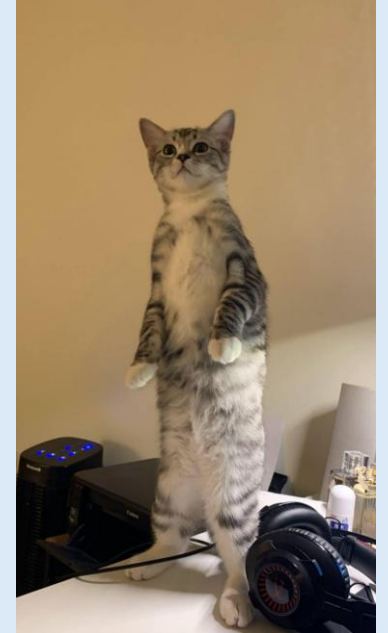
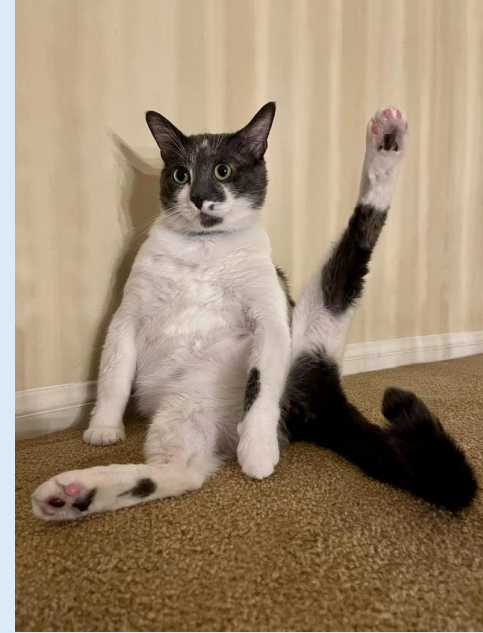
Challenges: background clutter



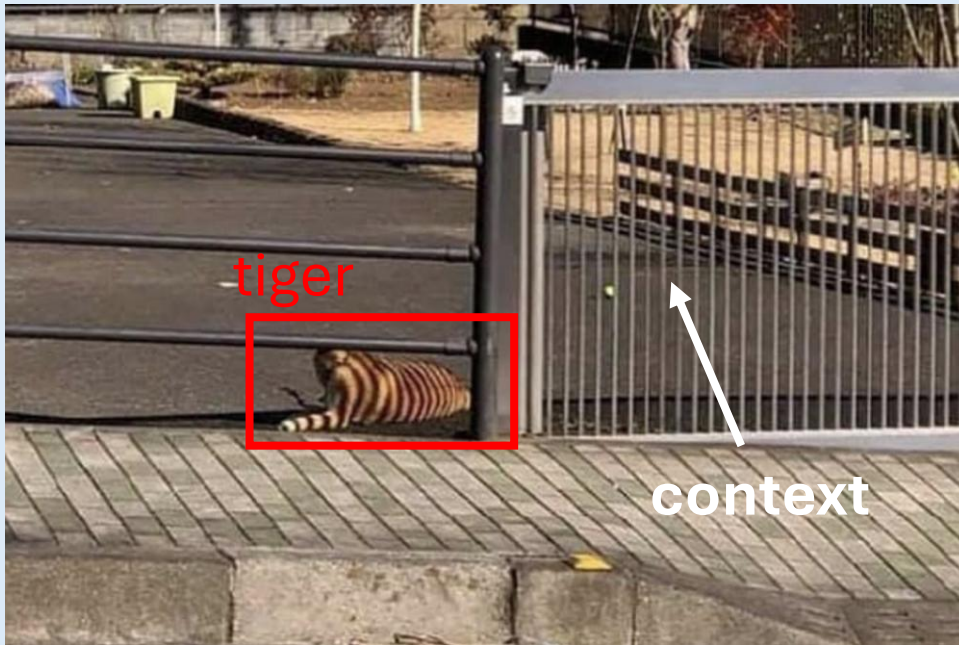
Challenges: occlusion



Challenges: deformation



Challenges: context



Challenges: intra class variation



Modern algorithm: learning from massive data



ImageNet: One of the most important datasets in computer vision. It consists of ~1M images.

An image classifier

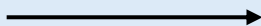
```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

Unlike e.g. sorting a list of numbers, there is **no obvious way** to hard-code the algorithm for recognizing a cat, or other classes.

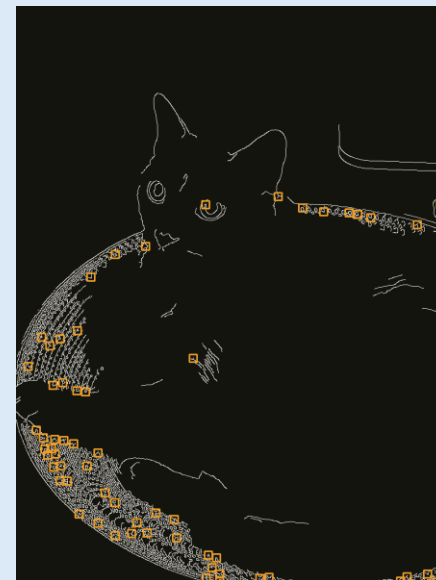
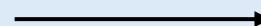
Attempts have been made



Find edges



Find corners



???

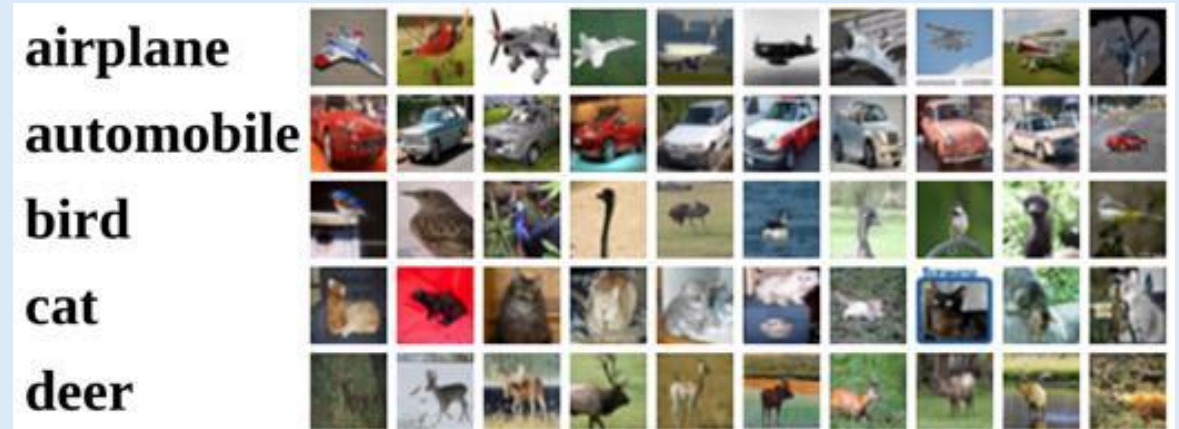
Machine learning: data-driven approach

1. Collect a dataset of images and labels
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images

```
def train(images, labels):  
    # Machine learning method  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

Example training set, CIFAR10:



Nearest Neighbor Classifier



First classifier: Nearest Neighbor

```
def train(images, labels):  
    # Machine learning method  
    return model
```



Memorize all data and labels

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



Predict the label of the most similar training image

First classifier: Nearest Neighbor

car



deer



cat



plane



bird



Training data with labels

???

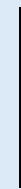
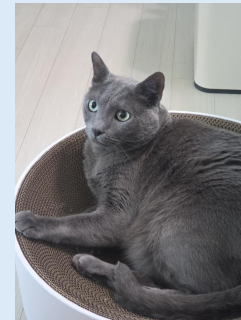


Query data

Distance metric



,



$\mathbb{R}$

Distance Metric to compare images

L1 distance: $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

—

=

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

add
→

456

First classifier: Nearest Neighbor

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

First classifier: Nearest Neighbor

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Memorize training data

First classifier: Nearest Neighbor

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

For each test image:

Find closest train image

Predict label of nearest image

First classifier: Nearest Neighbor

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

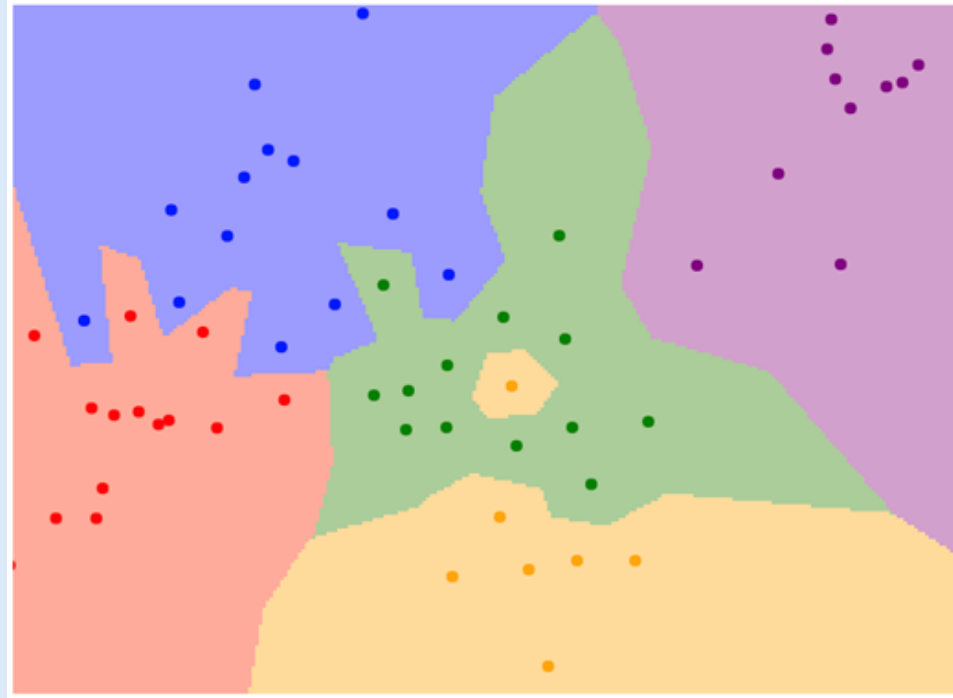
Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

Ans: Train $O(1)$, predict $O(N)$

This is bad: we want classifiers that are fast at prediction; slow for training is ok

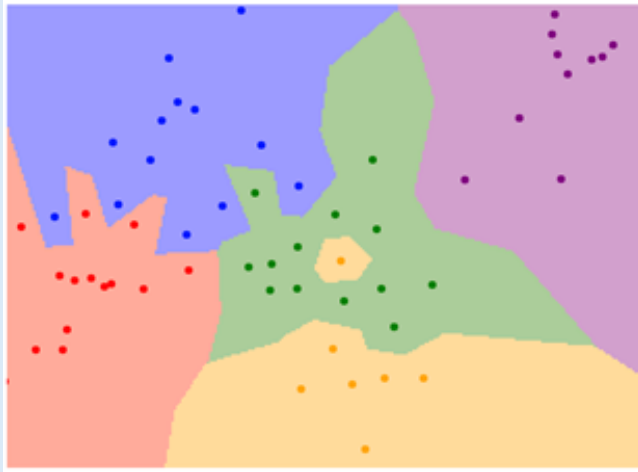
What does this look like?



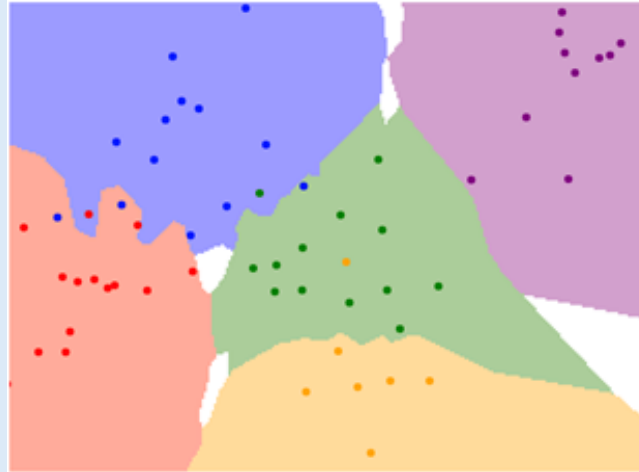
1-nearest neighbor

K-Nearest Neighbors

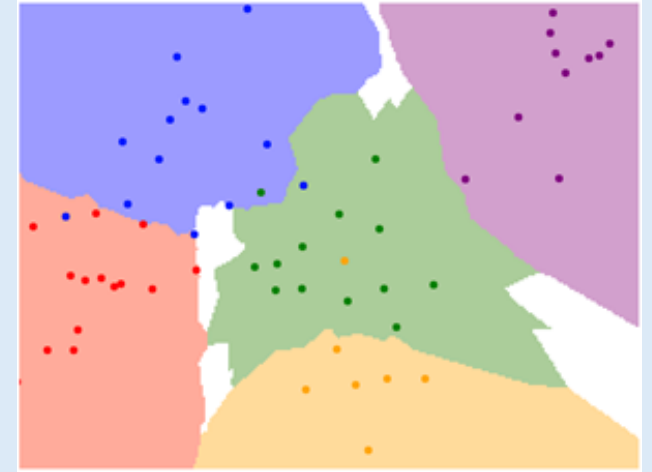
Instead of copying label from nearest neighbor, take majority vote from K closest points



$K = 1$



$K = 3$

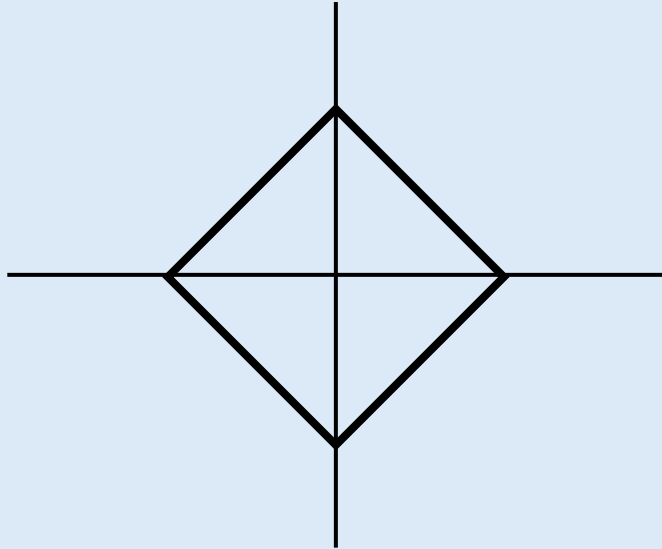


$K = 5$

K-Nearest Neighbors: Distance Metric

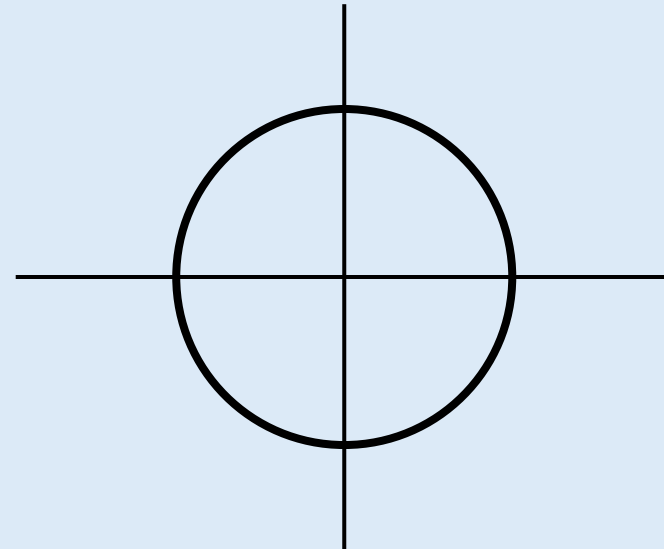
L1 (Manhattan) distance: Measures distance by moving along grid lines (like walking in a city with square blocks)

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



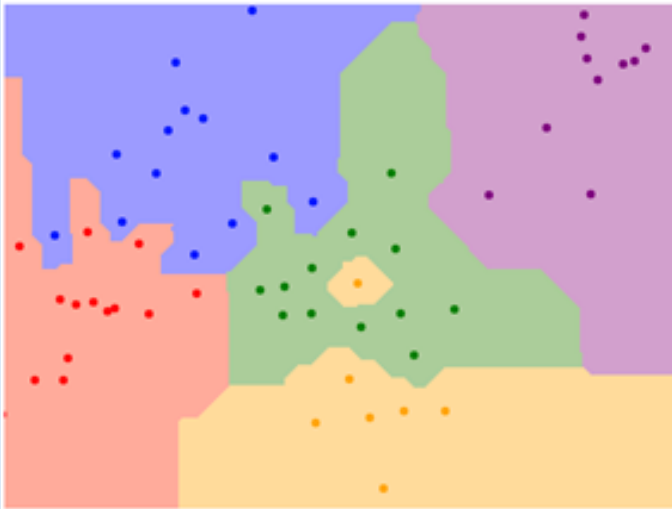
L2 (Euclidean) distance: Measures the straight-line distance (as the crow flies).

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

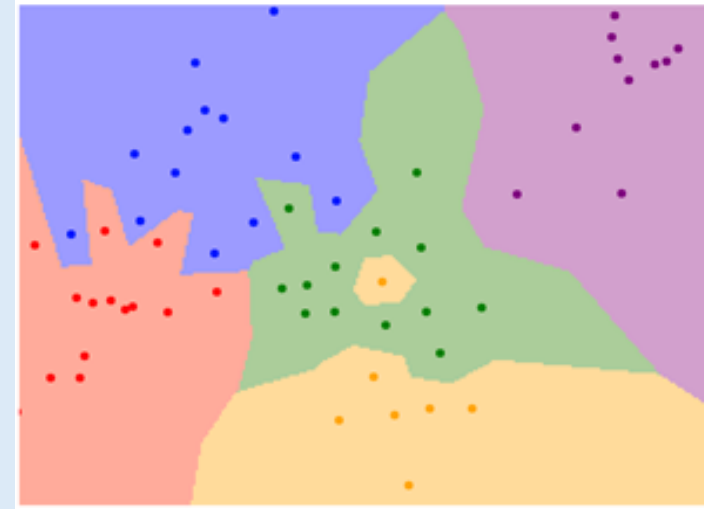


K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance



L2 (Euclidean) distance



Hyperparameters

What is the best value of k to use?

What is the best distance to use?

These are hyperparameters: choices about the algorithms themselves.

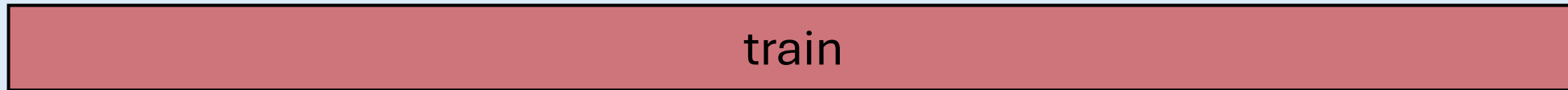
Very problem/dataset-dependent.

Must try them all out and see what works best.

Setting hyperparameters

Idea #1: choose hyperparameters that work best on the training data

BAD: $K = 1$ always works perfectly on training data



Idea #2: choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into train, val; choose hyperparameters on val and evaluate on test

Better!



Setting hyperparameters

train

Idea #4: Cross-Validation: Split data into folds, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5

test

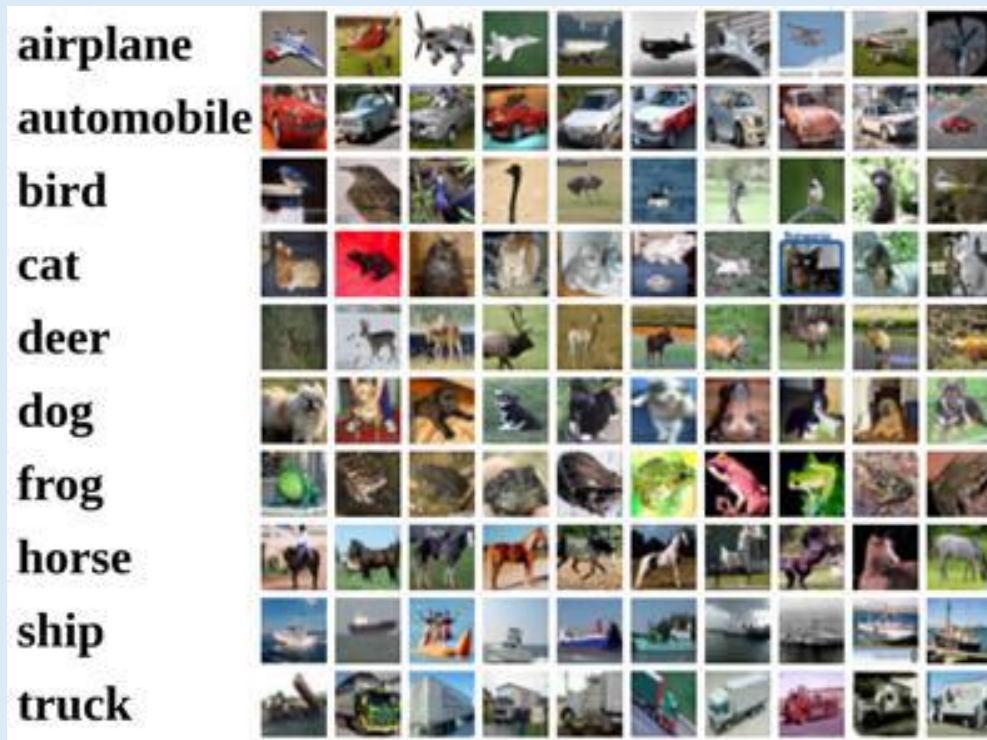
Useful for small datasets, but not used too frequently in deep learning

Example dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images

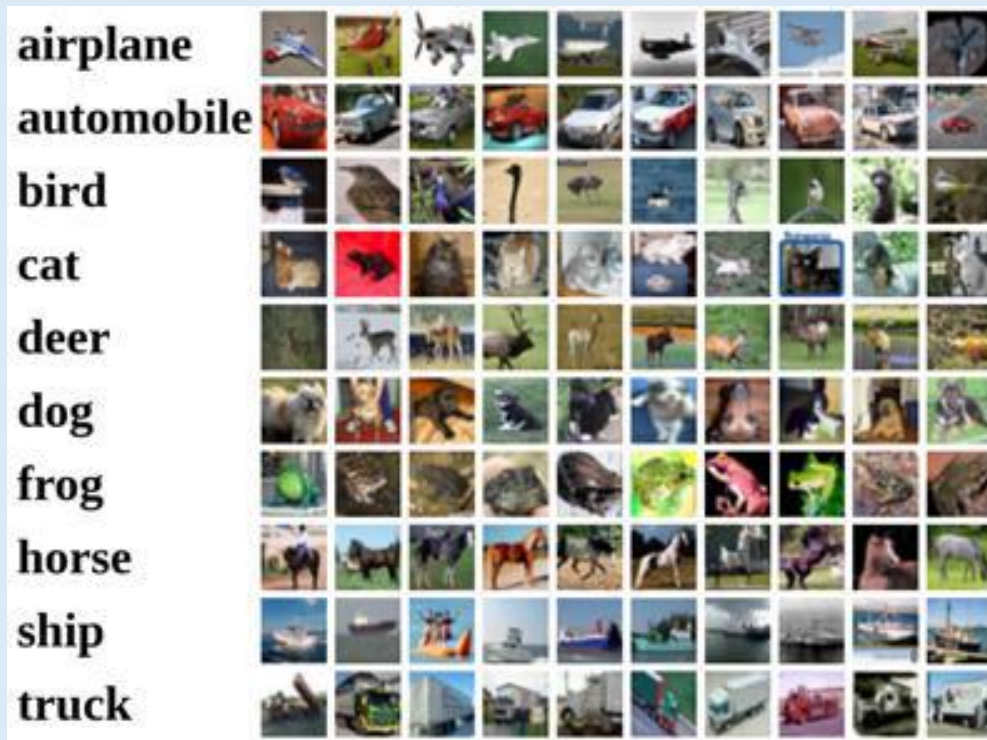


Example dataset: CIFAR10

10 classes

50,000 training images

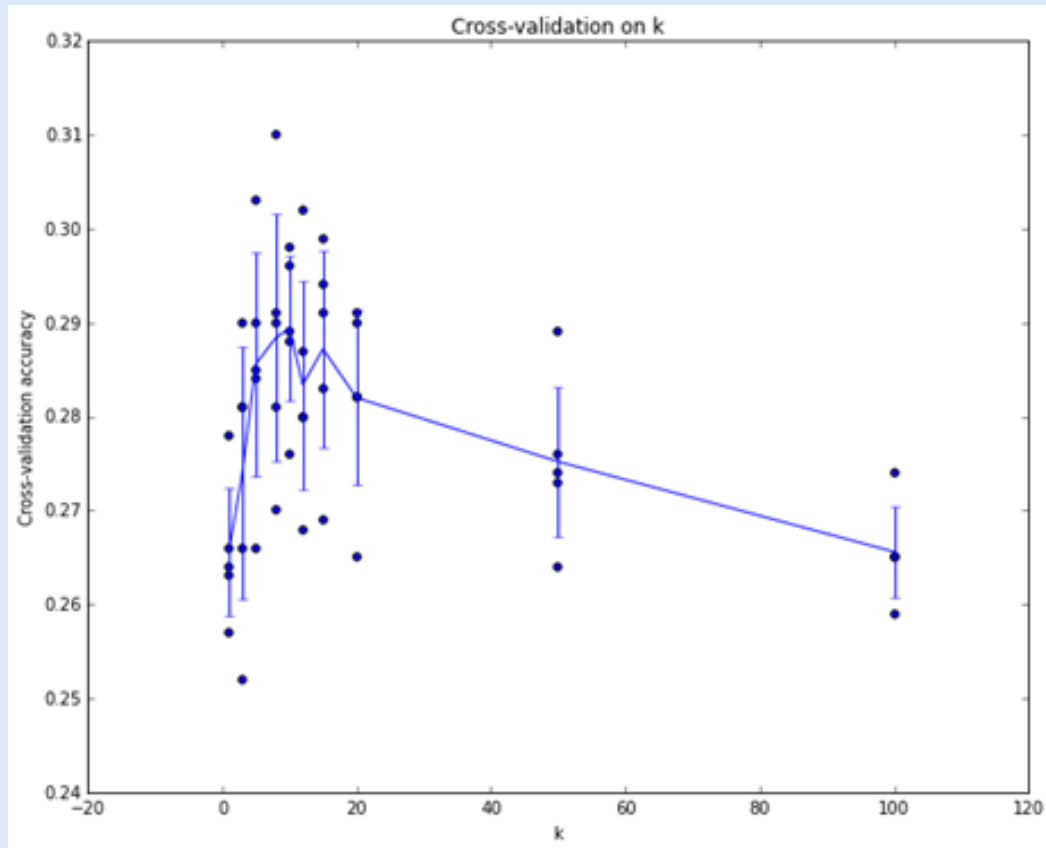
10,000 testing images



Test images and nearest neighbors



Setting hyperparameters



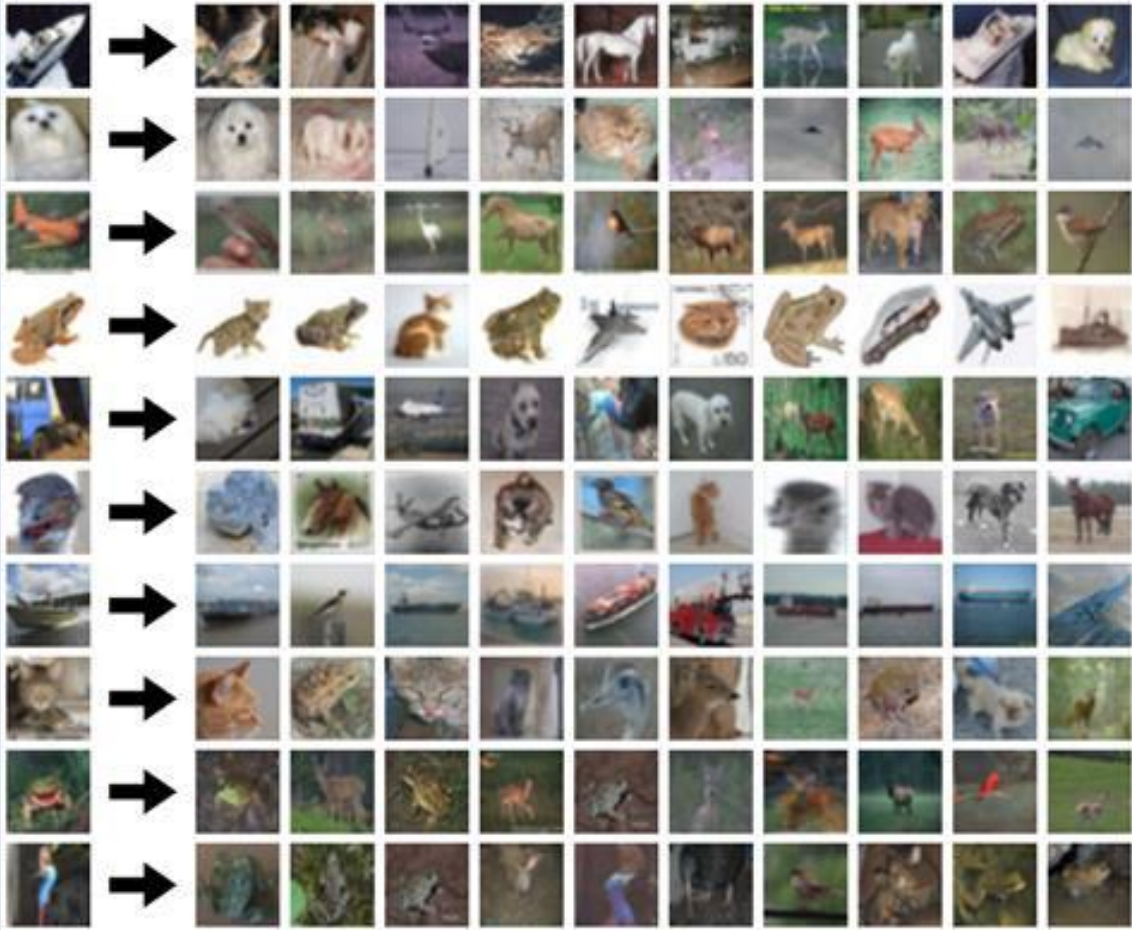
Example of 5-fold cross-validation for the value of k.

Each point: single outcome.

The line goes through the mean, bars indicated standard deviation.

(Seems that $k \approx 7$ works best for this data)

What does this look like?



What does this look like?



k-Nearest Neighbor with pixel distance is never used.

- Distance metrics on pixels are not informative

original



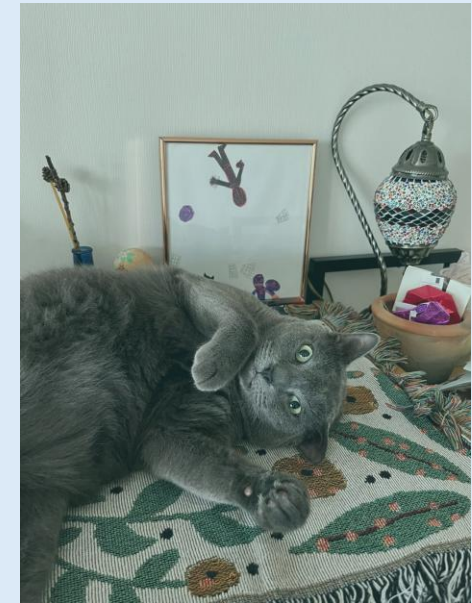
occluded



shifted (1 pixel)



tinted



All three images on the right have the same pixel distances to the one on the left!

K-Nearest Neighbors: Summary

- In image classification we start with a training set of images and labels, and must predict labels on the test set
- The K-Nearest Neighbors classifier predicts labels based on the **K nearest training examples**
- **Distance metric and K are hyperparameters**
- Choose hyperparameters using the validation set
- Only run on the test set once at the very end!

Linear Classifier



Parametric approach

Image



Array of 32x32x3 numbers
(3072 numbers total)

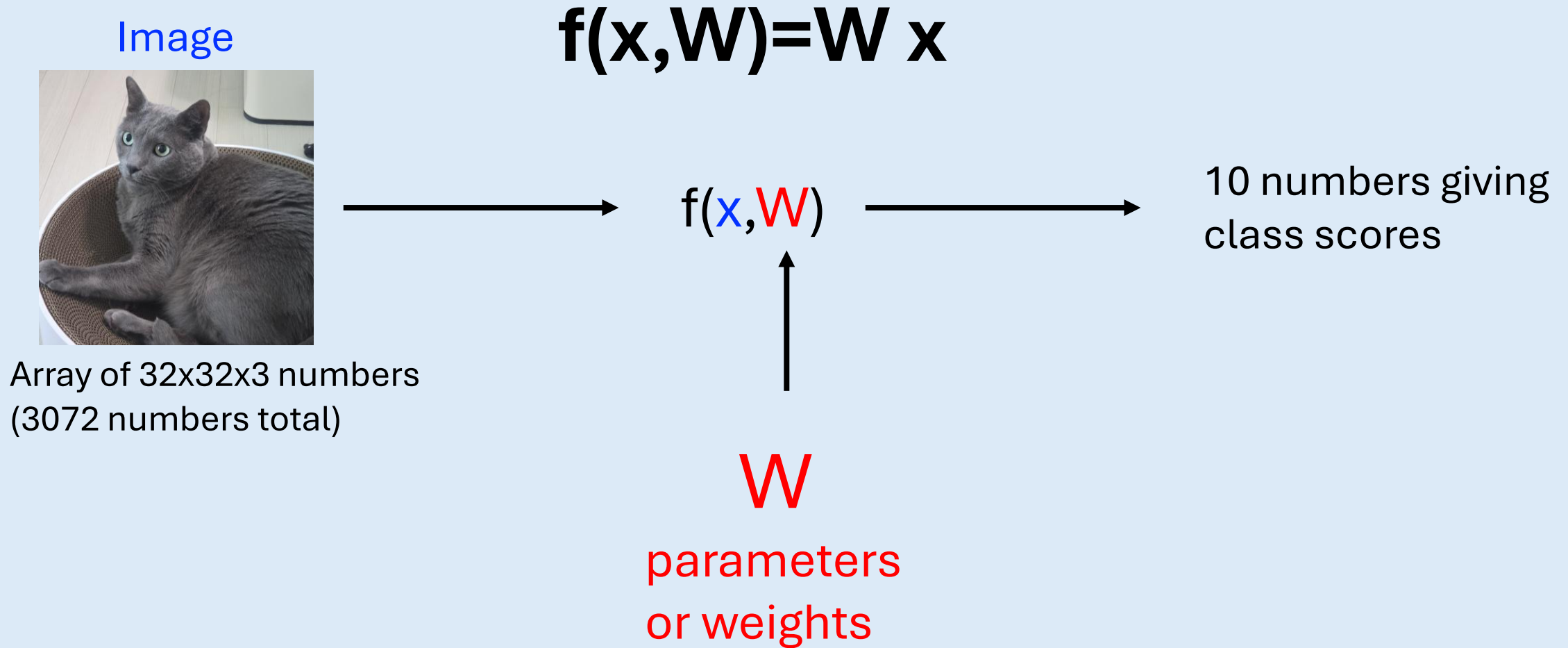
$f(x, W)$

10 numbers giving
class scores

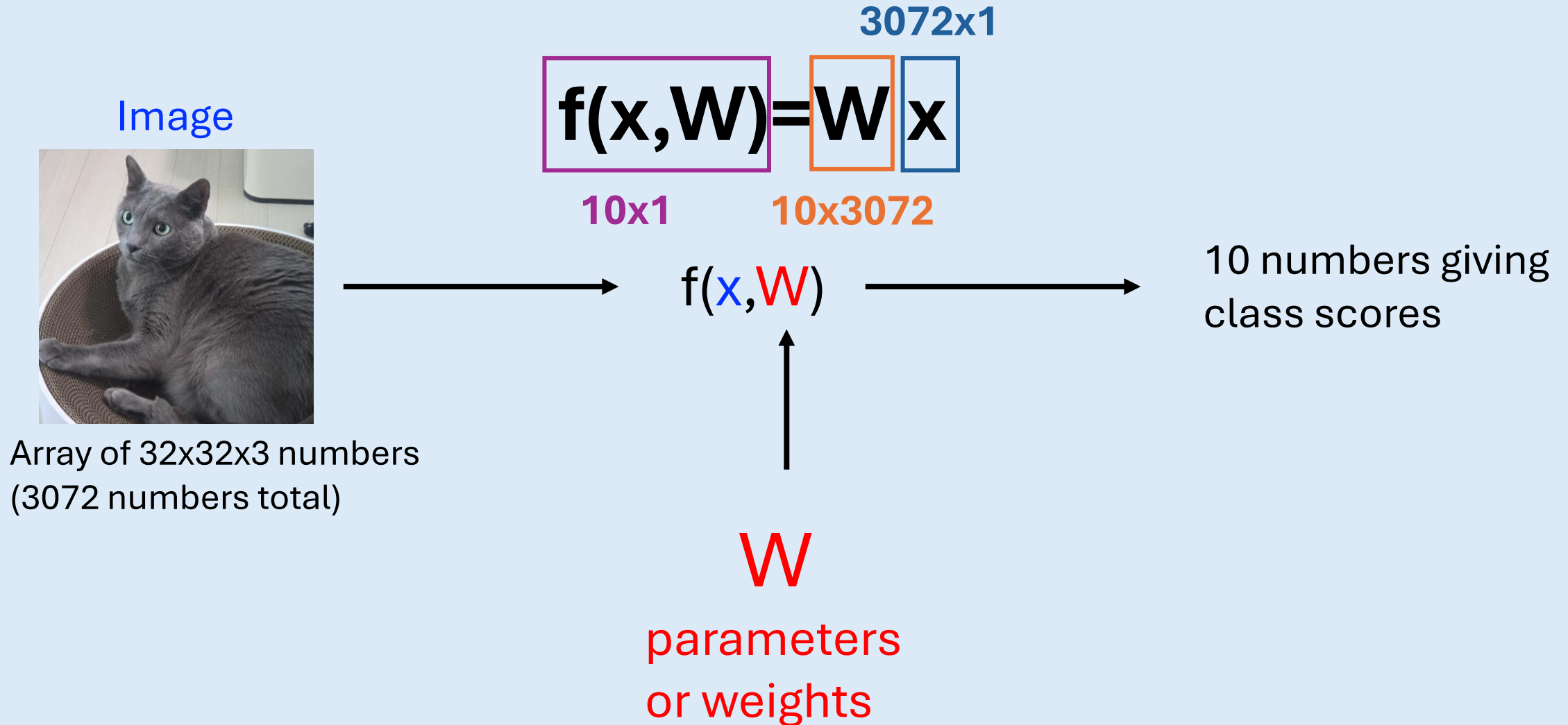
W

parameters
or weights

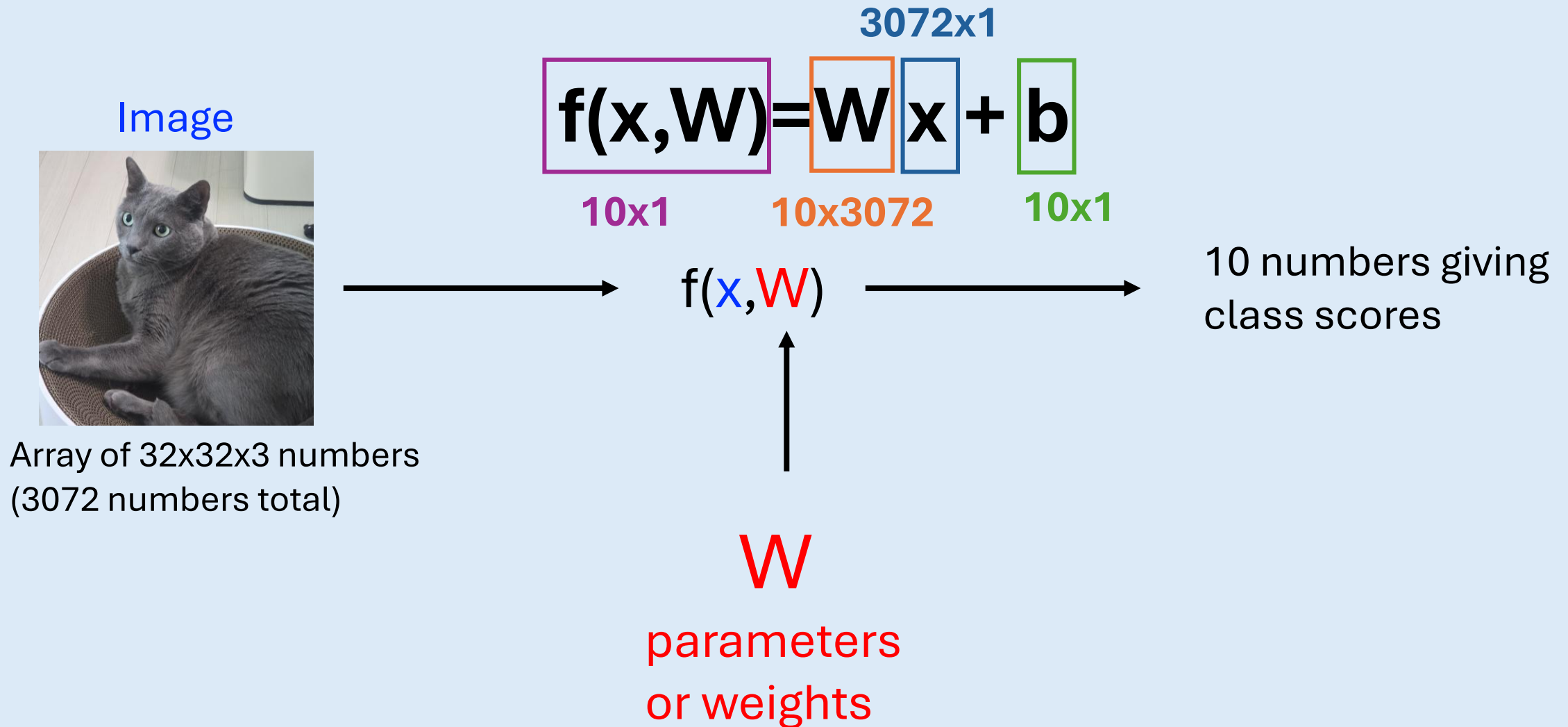
Parametric approach: Linear classifier



Parametric approach: Linear classifier

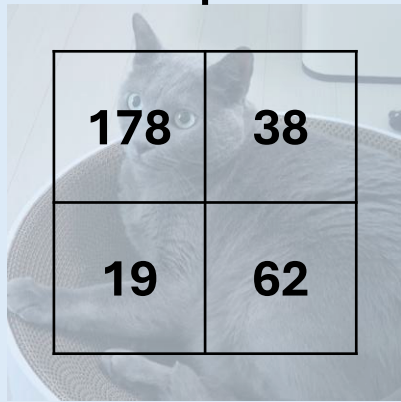


Parametric approach: Linear classifier

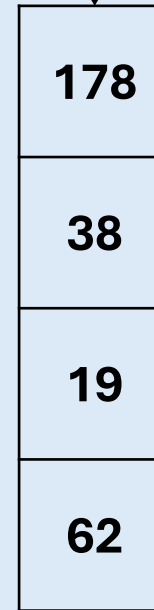


Example with an image with 4 pixels, and 3 classes (cat/dog/ship)

Flatten tensors into a vector

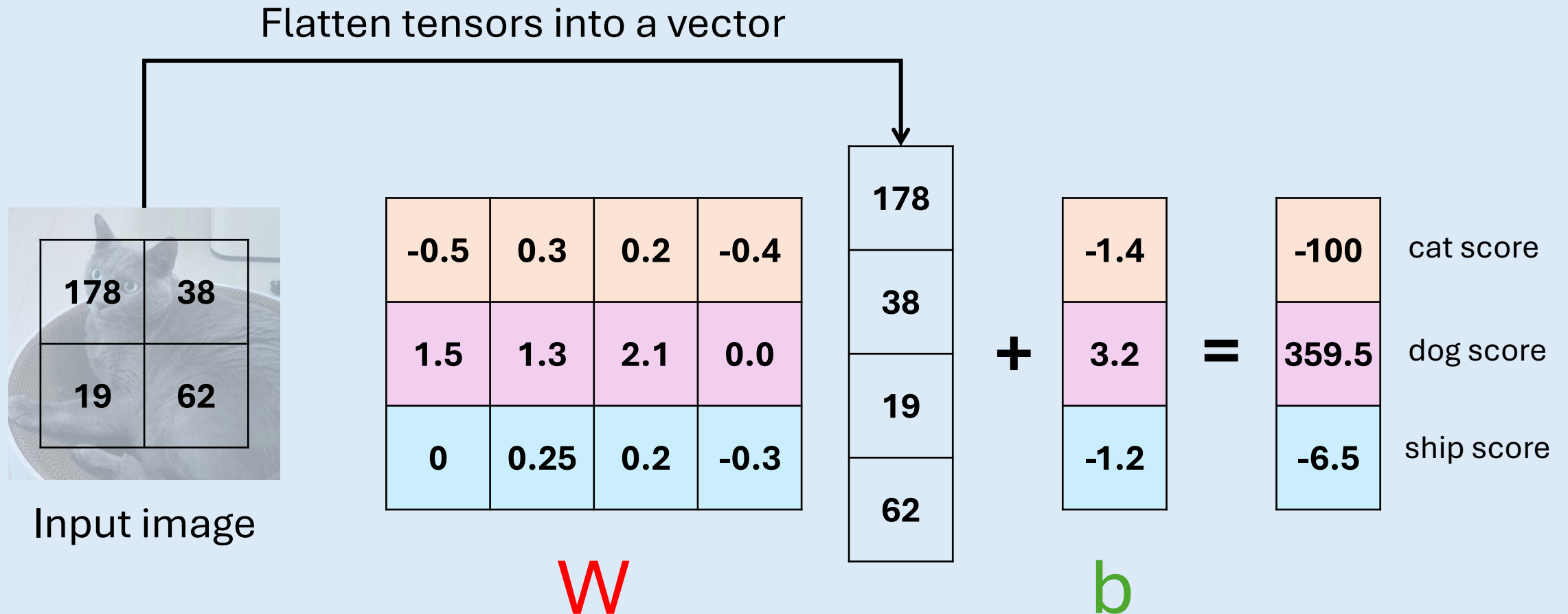


Input image

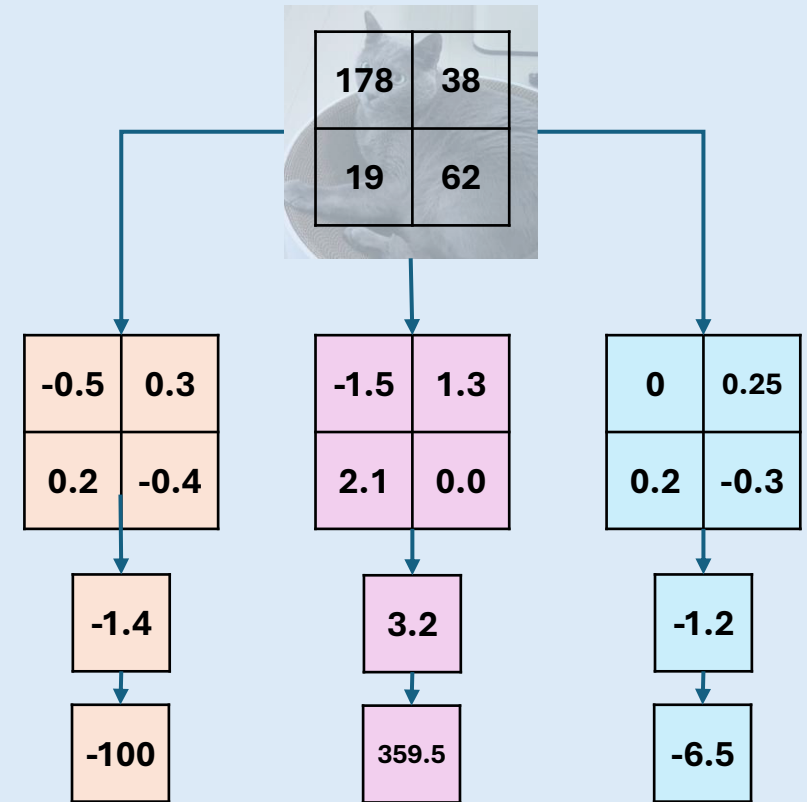
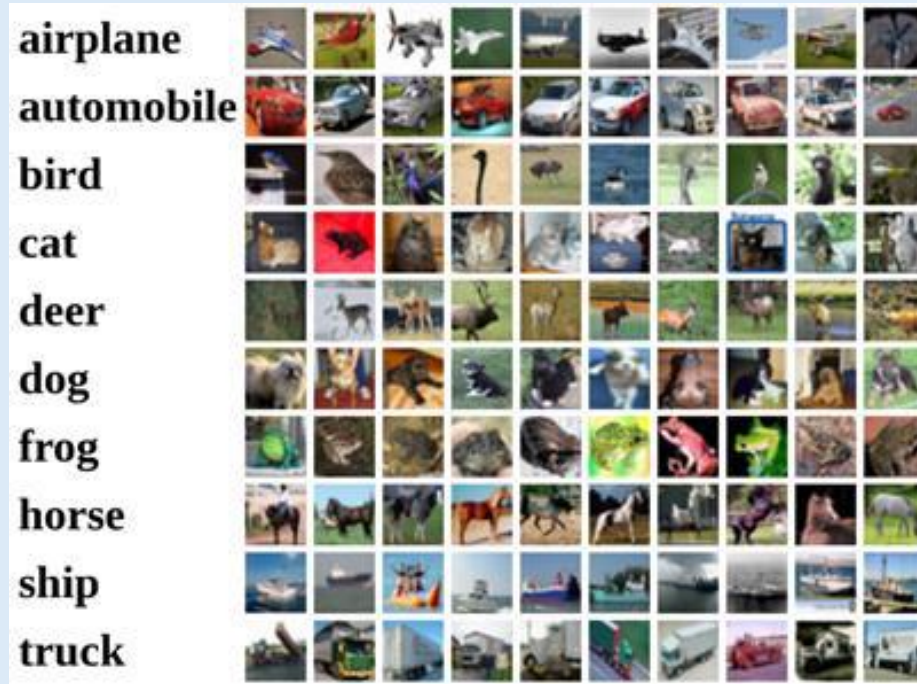


Example with an image with 4 pixels, and 3 classes (cat/dog/ship)

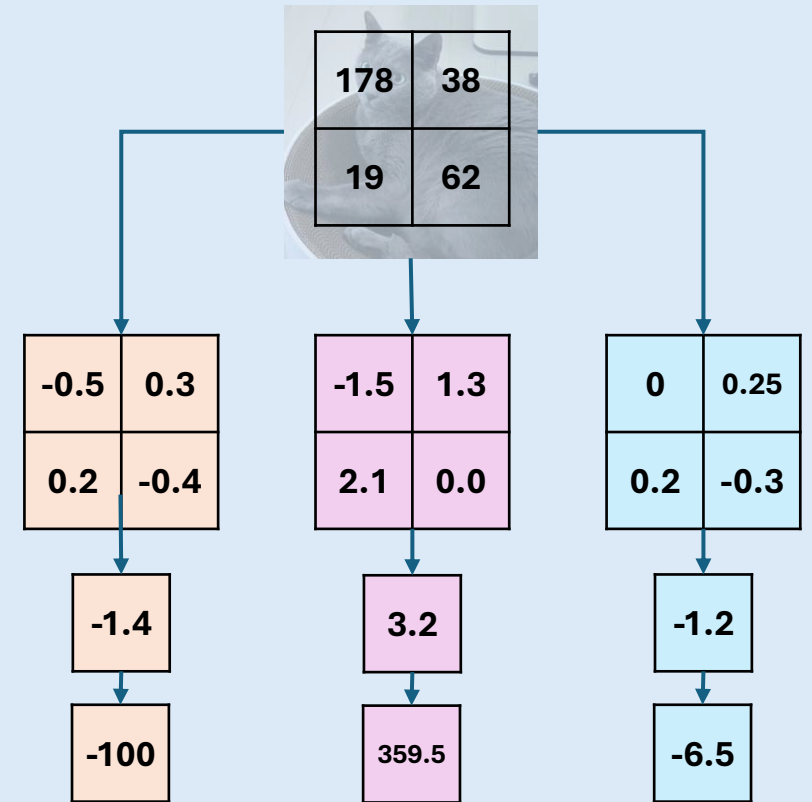
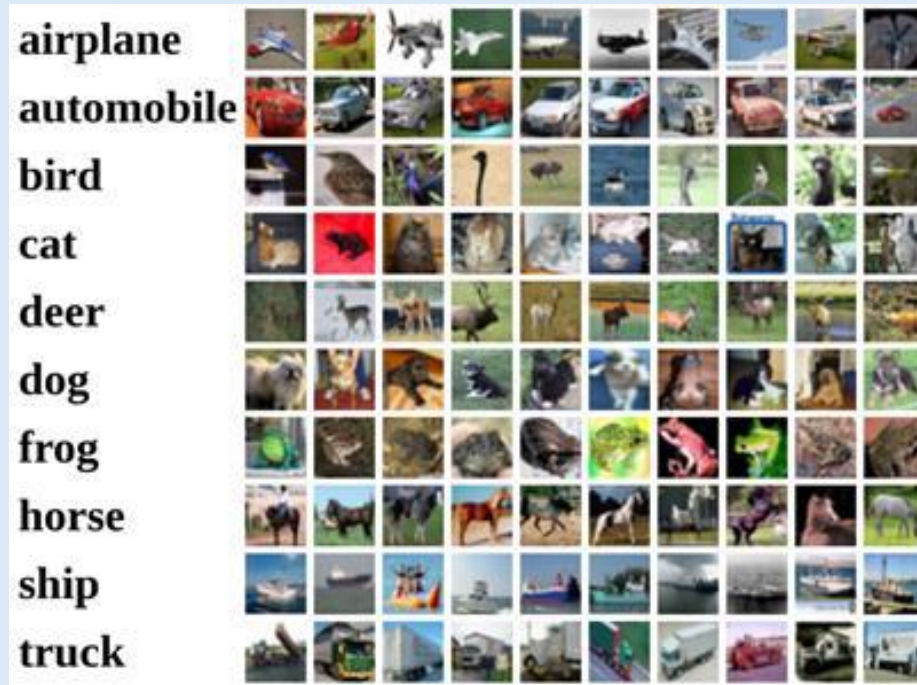
Algebraic viewpoint



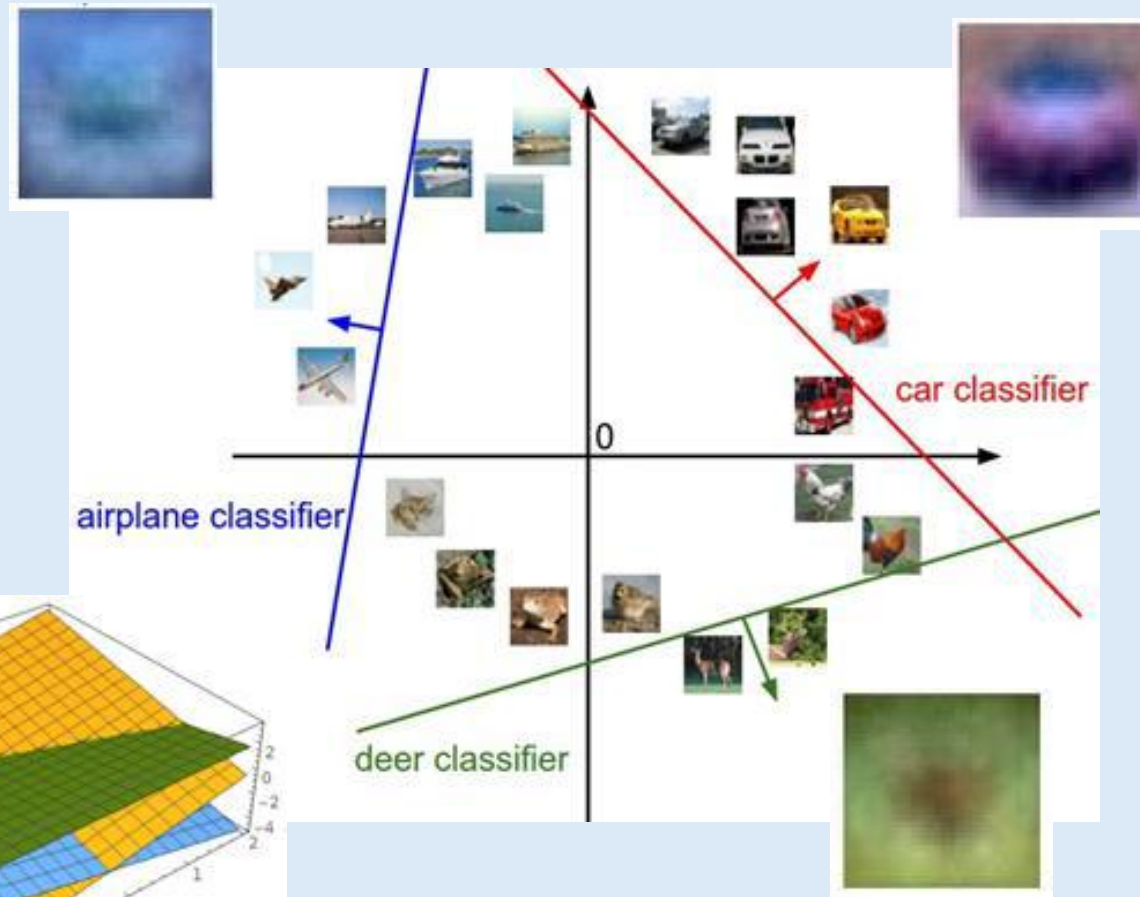
Interpreting a Linear Classifier



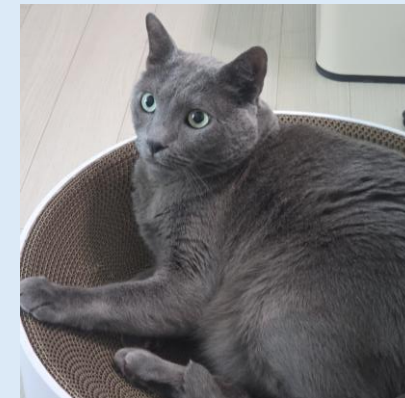
Interpreting a Linear Classifier: Visual Viewpoint



Interpreting a Linear Classifier: Geometric Viewpoint



$$f(x, W) = Wx + b$$

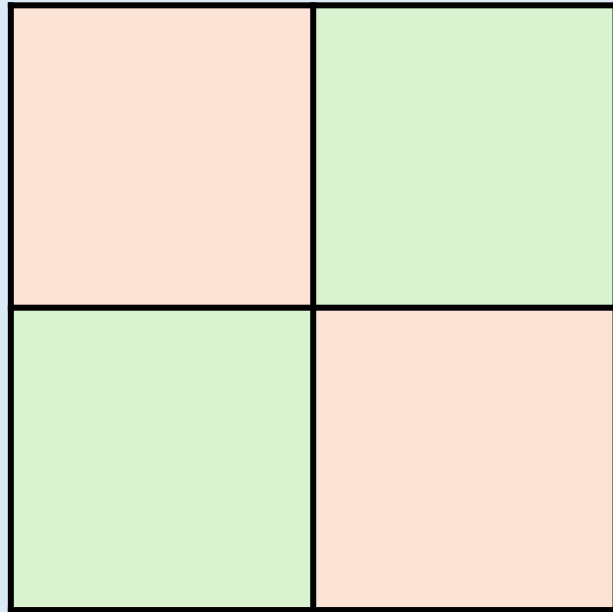


Array of 32x32x3 numbers
(3072 numbers total)

Hard cases for a linear classifier

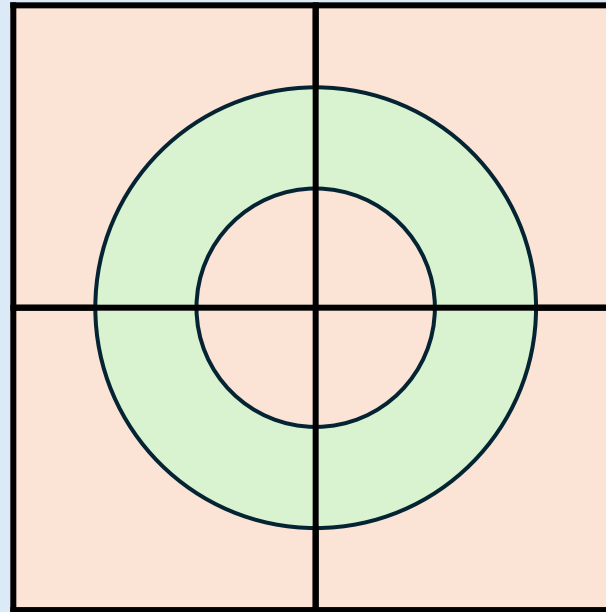
Class 1:

First and third quadrants



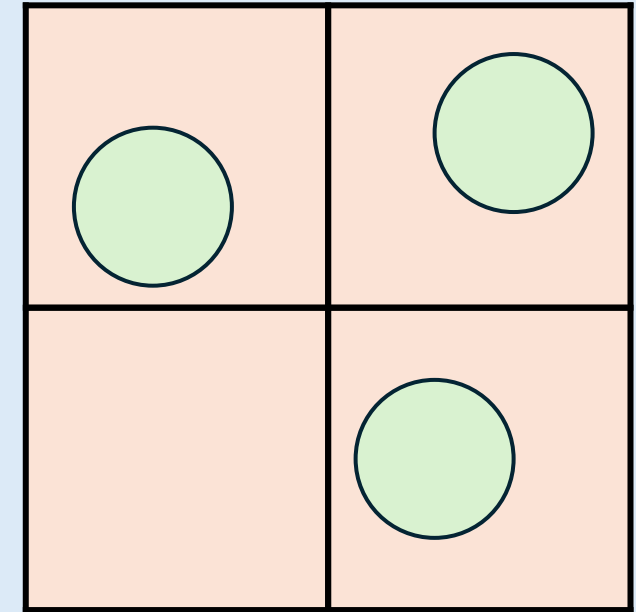
Class 1:

$1 \leq \text{L2 norm} \leq 2$



Class 1:

Three modes



Class 2:

Second and fourth quadrants

Class 2:

Everything else

Class 2:

Everything else

Linear classifier – choose a good W



<i>airplane</i>	-3.45	-0.51	3.42
<i>automobile</i>	-8.87	-2.09	4.64
<i>bird</i>	0.09	5.31	2.65
<i>cat</i>	2.9	-4.22	5.1
<i>deer</i>	4.48	-4.19	2.64
<i>dog</i>	8.02	3.58	-4.34
<i>frog</i>	3.78	4.49	5.55
<i>horse</i>	1.06	-4.37	-1.5
<i>ship</i>	-0.36	6.04	-4.79
<i>truck</i>	-0.72	-2.93	6.14

1. Define a loss function that quantifies our unhappiness with the scores across the training data.
2. Come up with a way of efficiently finding the parameters that minimize the loss function.
(optimization)

Linear classifier – choose a good W



<i>airplane</i>	-3.45	-0.51	3.42
<i>automobile</i>	-8.87	-2.09	4.64
<i>bird</i>	0.09	5.31	2.65
<i>cat</i>	2.9	-4.22	5.1
<i>deer</i>	4.48	-4.19	2.64
<i>dog</i>	8.02	3.58	-4.34
<i>frog</i>	3.78	4.49	5.55
<i>horse</i>	1.06	-4.37	-1.5
<i>ship</i>	-0.36	6.04	-4.79
<i>truck</i>	-0.72	-2.93	6.14

$$f(x, W) = W x$$

A loss function tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

Where x_i is image and y_i is (integer) label

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Softmax Classifier



Softmax Classifier (Multinomial Logistic Regression)

Want to interpret raw classifier scores as probabilities



cat	3.2
dog	5.1
ship	-1.7

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

cat	3.2
dog	5.1
ship	-1.7

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

cat	3.2		24.5
dog	5.1	→	164.0
ship	-1.7		0.18

unnormalized
probabilities

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

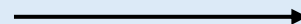
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

cat	3.2
dog	5.1
ship	-1.7



24.5
164.0
0.18

normalize

0.13
0.87
0.00

unnormalized
probabilities

probabilities

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

cat

3.2

dog

5.1

ship

-1.7

Unnormalized log-
probabilities / logits

24.5

164.0

0.18

unnormalized
probabilities

normalize

0.13

0.87

0.00

probabilities

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

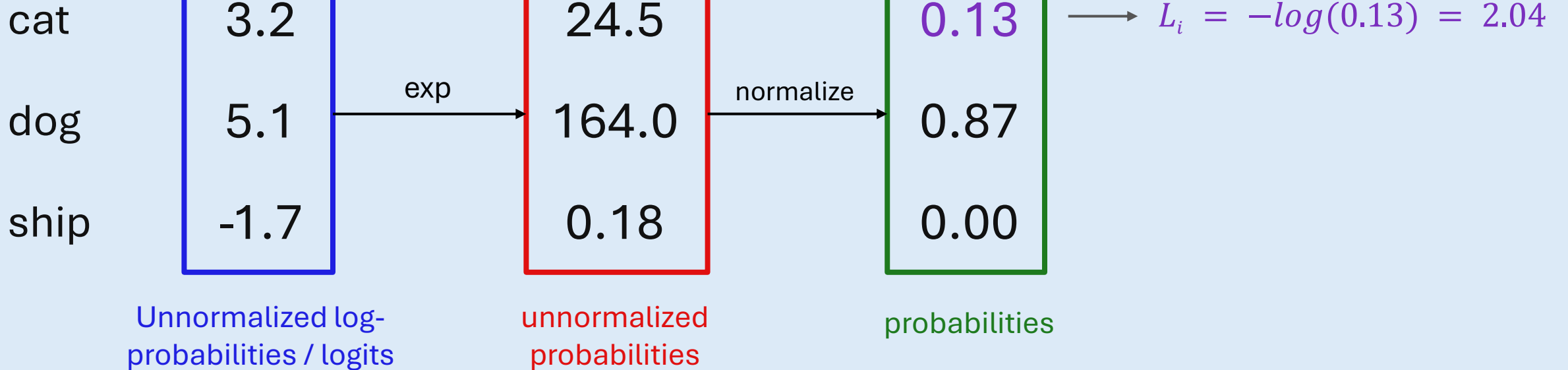
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

$$L_i = -\log P(Y = y_i | X = x_i)$$



Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

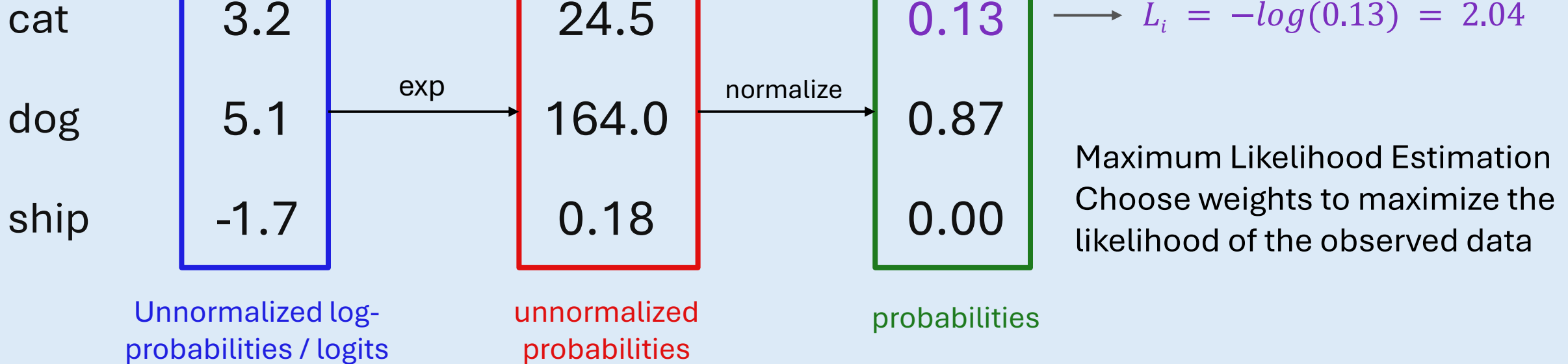
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

$$L_i = -\log P(Y = y_i | X = x_i)$$



Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

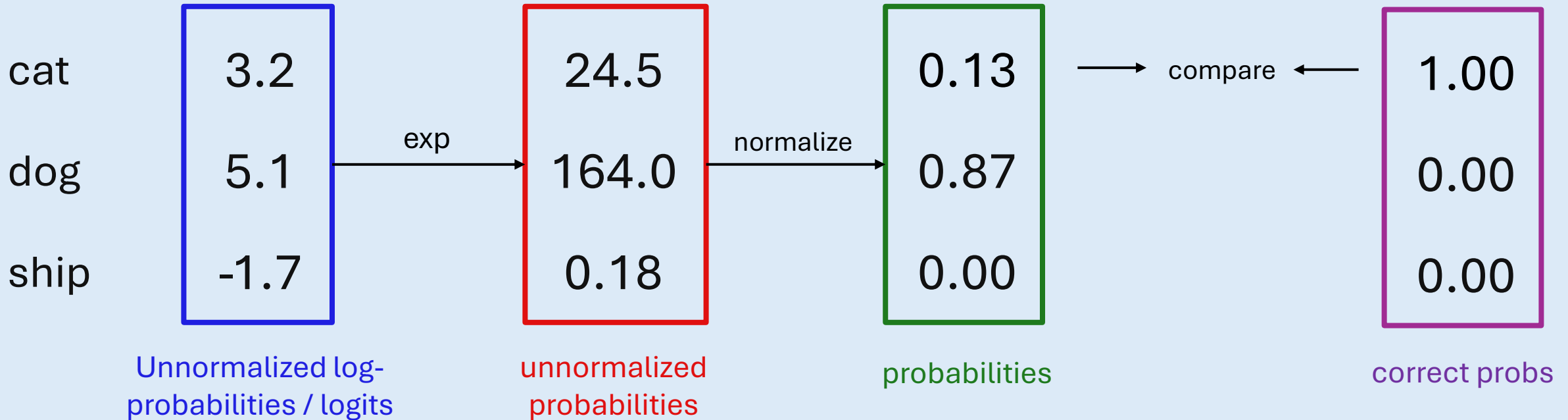
$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Probabilities
must be ≥ 0

Probabilities
must sum to 1

$$L_i = -\log P(Y = y_i | X = x_i)$$



Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

Putting it all together:

cat 3.2

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log \frac{e^{s_i}}{\sum_j e^{s_j}}$$

dog 5.1

ship -1.7

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

Putting it all together:

cat 3.2

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log \frac{e^{s_i}}{\sum_j e^{s_j}}$$

dog 5.1

ship -1.7

Q1: What is the min/max possible softmax loss L_i ?

Q2: At initialization all s_j will be approximately equal; what is the softmax loss L_i , assuming C classes?

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$$s = f(x_i; W)$$

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Softmax
Function

Maximize probability of correct class

Putting it all together:

cat 3.2

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log \frac{e^{s_i}}{\sum_j e^{s_j}}$$

dog 5.1

ship -1.7

Q2: At initialization all s will be approximately equal; what is the loss?

A: $-\log(1/C) = \log(C)$, if $C = 10$, then $L_i = \log(10) \approx 2.3$

References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

